Checksum receiver side

```
a = input("Enter first 8-bit data: ")
b = input("Enter second 8-bit data: ")

sum_value = int(a, 2) + int(b, 2)

while sum_value > 255:
    sum_value = (sum_value & 255) + (sum_value >> 8)

checksum = 255 - sum_value

print("Checksum:", format(checksum, '08b'))

receiver_sum = sum_value + checksum

while receiver_sum > 255:
    receiver_sum = (receiver_sum & 255) + (receiver_sum >> 8)

print("Receiver result:", format(receiver_sum, '08b'))

if receiver_sum == 255:
    print("No error detected")
else:
    print("Error detected")
```

EXAMPLE

Enter first 8-bit data: 10011001

Enter second 8-bit data: 11100010

Checksum: 10000100

Receiver result: 11111111

No error detected

1. What is the problem asking?

We need to write a Python program to implement the Checksum error detection technique.

Main idea: Sender sends data to the receiver. During transmission, data may get corrupted. Checksum helps detect whether an error occurred.

2. Understand the given input

Enter first 8-bit data: 10011001

Enter second 8-bit data: 11100010

We have two binary numbers:

- First data = 10011001
- Second data = 11100010

Our task is to calculate the checksum.

3. Step-by-step calculation

Step 1: Add both binary numbers.

```
  10011001
+ 11100010
----------
 101111011
```

The result is 9 bits. But we need only 8 bits.

Step 2: Add the extra carry to the remaining 8 bits.

```
  01111011
+       1
----------
  01111100
```

Step 3: Find 1's complement.

Change every 0 to 1 and every 1 to 0.

Sum : 01111100

Checksum : 10000011

Step 4: Receiver verification.

01111100

+ 10000011

11111111

Sender side

```
a = input("Enter first 8-bit data: ")
b = input("Enter second 8-bit data: ")

sum_value = int(a, 2) + int(b, 2)

while sum_value > 255:
    sum_value = (sum_value & 255) + (sum_value >> 8)

checksum = 255 - sum_value

print("First data:", a)
print("Second data:", b)
print("Checksum:", format(checksum, '08b'))

print("Transmitted data:", a, b, format(checksum, '08b'))
```

output

Enter first 8-bit data: 10011001

Enter second 8-bit data: 11100010

First data: 10011001

Second data: 11100010

Checksum: 10000011

Transmitted data: 10011001 11100010 10000011

Example

Checksum – Sender Side Example

Input:

First data = 10011001

Second data = 11100010

Step 1: Add both binary numbers

```
  10011001
+ 11100010
 ----------
 101111011
```

Step 2: Add the carry (1) to the remaining 8 bits

```
  01111011
+       1
 ----------
  01111100
```

Step 3: Find 1's complement

Change 0 to 1 and 1 to 0.

Sum   = 01111100

Checksum = 10000011

Step 4: Transmitted data

First data : 10011001

Second data : 11100010

Checksum   : 10000011

Final checksum: 10000011

File handling

```
# Writing into a file
f = open("sample.txt", "w")
f.write("Hello Networking")
f.close()

# Reading the file
f = open("sample.txt", "r")
print("File content:", f.read())
f.close()

# Appending data
f = open("sample.txt", "a")
f.write("\nComputer Networks")
f.close()

# Reading updated file
f = open("sample.txt", "r")
print("Updated content:")
print(f.read())
f.close()
```

output

```
File content: Hello Networking

Updated content:
Hello Networking
Computer Networks
```

7. Routing Table

Aim: To implement a simple routing table.

Theory: A routing table stores information about destinations and the next hop used to forward packets.

Python Program:

```
routing_table = {
   "192.168.1.1": "Router A",
   "192.168.1.2": "Router B",
   "192.168.1.3": "Router C"
}

ip = input("Enter destination IP: ")

if ip in routing_table:
   print("Next hop:", routing_table[ip])
else:
   print("Destination not found")
```

output

Enter destination IP: 192.168.1.2

Next hop: Router B

WEEK 14: Dijkstra's Shortest Path Algorithm

1. Aim

To implement Dijkstra's algorithm to find the shortest path from a source vertex to all other vertices in a weighted graph.

2. Theory

1. Dijkstra's algorithm finds the shortest path in a graph.
2. It uses the greedy technique.
3. It finds the minimum distance from a source vertex.
4. It selects the nearest unvisited vertex.
5. It updates the distances of neighbouring vertices.
6. It works only with non-negative edge weights.

3. Python Program

```
n = int(input("Enter number of vertices: "))

print("Enter adjacency matrix:")

graph = []

for i in range(n):
    row = list(map(int, input().split()))
    graph.append(row)

src = int(input("Enter source vertex: "))

dist = [9999] * n
visited = [False] * n

dist[src] = 0

for count in range(n):
```

```
    min_dist = 9999
    u = -1

    for i in range(n):
        if not visited[i] and dist[i] < min_dist:
            min_dist = dist[i]
            u = i

    if u == -1:
        break

    visited[u] = True

    for v in range(n):
        if (not visited[v] and graph[u][v] != 0
            and dist[u] + graph[u][v] < dist[v]):

            dist[v] = dist[u] + graph[u][v]

print("Shortest distances from source", src)

for i in range(n):
    if dist[i] == 9999:
        print("To vertex", i, "= Unreachable")
    else:
        print("To vertex", i, "=", dist[i])
```

4. Sample Input

```
Enter number of vertices: 4

Enter adjacency matrix:
```

0 2 4 0

2 0 1 7

4 1 0 3

0 7 3 0

Enter source vertex: 0

5. Sample Output

Shortest distances from source 0

To vertex 0 = 0

To vertex 1 = 2

To vertex 2 = 3

To vertex 3 = 6

6. Explanation of Output

Starting vertex = 0

Path	**Distance**
0 → 0	0
0 → 1	2
0 → 1 → 2	3
0 → 1 → 2 → 3	6

The shortest distance to vertex 3 is 6 because 2 + 1 + 3 = 6.

7. Important Variables

Variable	**Purpose**
n	Number of vertices
graph	Stores adjacency matrix
src	Starting vertex
dist	Stores shortest distances
visited	Tracks visited vertices
min_dist	Minimum distance

u Selected vertex

8. Viva Questions

1. **What is Dijkstra's algorithm?**
   It is a shortest path algorithm.
2. **Which technique is used?**
   Greedy technique.
3. **What is a vertex?**
   A node in a graph.
4. **What is an edge?**
   Connection between two vertices.
5. **Can Dijkstra handle negative weights?**
   No.
6. **What is an adjacency matrix?**
   A matrix used to represent connections between vertices.
7. **What is the source vertex?**
   The starting vertex.

External Lab Tip

Remember these three important lines:

dist[src] = 0

visited[u] = True

dist[v] = dist[u] + graph[u][v]

Basic Networking Commands

1. IPConfig Command

Theory:

- IPConfig stands for Internet Protocol Configuration.
- It is used to display the IP configuration of a computer.
- It shows IP address, subnet mask and default gateway.

Syntax:

ipconfig

Output:

Ethernet adapter Wi-Fi:

IPv4 Address . . . . . : 192.168.1.10

Subnet Mask . . . . . : 255.255.255.0

Default Gateway . . . . . : 192.168.1.1

Explanation of Output:

- IPv4 Address: Identifies the computer on the network.
- Subnet Mask: Identifies the network and host portions of the IP address.
- Default Gateway: Router address used to communicate with other networks.

2. Ping Command

Theory:

- Ping stands for Packet Internet Groper.
- It is used to check network connectivity.
- It sends packets to the destination and waits for replies.
- It helps identify connection problems.

Syntax:

ping google.com

Output:

Pinging google.com

Reply from 142.250.183.14:

bytes=32 time=20ms TTL=116

Packets: Sent = 4

Received = 4

Lost = 0

Explanation of Output:

- Bytes: Size of the data packet.
- Time: Time taken to receive a reply.
- TTL: Time To Live.
- Sent: Number of packets sent.
- Received: Number of packets received.
- Lost: Number of packets lost.

3. Tracert Command

Theory:

- Tracert stands for Trace Route.
- It is used to find the path taken by data packets.
- It displays intermediate routers between source and destination.
- It helps identify network delays.

Syntax:

tracert google.com

Output:

Tracing route to google.com

1 192.168.1.1

2 10.10.0.1

3 142.250.183.14

Trace complete.

Explanation of Output:

- First hop: Local router.
- Second hop: Next router in the network.
- Third hop: Destination network.

- Trace complete: Route tracing is finished.

4. Netstat Command

Theory:

- Netstat stands for Network Statistics.
- It displays active network connections.
- It shows TCP connections and listening ports.
- It is useful for monitoring network activity.

Syntax:

netstat

Output:

Proto Local Address Foreign Address State

TCP 192.168.1.10:5000 142.250.1.1:443 ESTABLISHED

TCP 192.168.1.10:5001 192.168.1.1:80 TIME_WAIT

Explanation of Output:

- Proto: Network protocol (TCP/UDP).
- Local Address: Your computer's IP and port.
- Foreign Address: Remote computer's IP and port.
- State: Current connection status.
- ESTABLISHED: Connection is active.

5. Hostname Command

Theory:

- Hostname displays the name of the computer.
- It helps identify a computer in a network.
- Every computer can have a hostname.

Syntax:

hostname

Output:

DESKTOP-ABC123

Explanation of Output:

- DESKTOP-ABC123 is the name of the computer.
- The output identifies the system hostname.

6. Nslookup Command

Theory:

- Nslookup stands for Name Server Lookup.
- It is used to find the IP address of a domain name.
- It checks DNS information.
- It helps troubleshoot DNS problems.

Syntax:

nslookup google.com

Output:

Server: dns.google

Address: 8.8.8.8

Name: google.com

Address: 142.250.183.14

Explanation of Output:

- Server: DNS server name.
- Address: DNS server IP address.
- Name: Domain name being searched.
- Address: IP address associated with the domain.

7. ARP Command

Theory:

- ARP stands for Address Resolution Protocol.
- It is used to map IP addresses to MAC addresses.
- It maintains an ARP table.
- It helps devices communicate within a local network.

Syntax:

arp -a

Output:

Internet Address Physical Address Type

192.168.1.1 AA-BB-CC-DD-EE-FF dynamic

192.168.1.2 11-22-33-44-55-66 dynamic

Explanation of Output:

- Internet Address: IP address of a device.
- Physical Address: MAC address of that device.
- Type: Dynamic or static mapping.

8. Getmac Command

Theory:

- Getmac displays the MAC address of a computer.
- MAC stands for Media Access Control.
- It identifies the network interface hardware.

Syntax:

getmac

Output:

Physical Address Transport Name

AA-BB-CC-DD-EE-FF \Device\Tcpip_{123}

Explanation of Output:

- Physical Address: MAC address of the network adapter.
- Transport Name: Network interface identification.

9. Pathping Command

Theory:

- Pathping is a network troubleshooting command.
- It combines the features of Ping and Tracert.
- It identifies packet loss and network delays.
- It helps find problems along the network path.

Syntax:

pathping google.com

Output:

Tracing route to google.com

0 DESKTOP-ABC123

1 192.168.1.1

2 10.10.0.1

3 142.250.183.14

Computing statistics...

Explanation of Output:

- Displays the routers along the path.
- Computes packet loss at different hops.
- Helps identify network problems.

10. IPConfig /all Command

Theory:

- It displays complete network configuration details.
- It shows IP address, MAC address and DNS server.
- It also displays DHCP information.
- It is useful for detailed network troubleshooting.

Syntax:

ipconfig /all

Output:

Host Name : DESKTOP-ABC123

Physical Address : AA-BB-CC-DD-EE-FF

IPv4 Address : 192.168.1.10

Subnet Mask : 255.255.255.0

DHCP Enabled : Yes

Default Gateway : 192.168.1.1

DNS Servers : 8.8.8.8

Explanation of Output:

- Host Name: Computer name.
- Physical Address: MAC address.
- IPv4 Address: Computer's IP address.
- Subnet Mask: Divides network and host portions.
- DHCP Enabled: Indicates automatic IP configuration.
- Default Gateway: Router IP address.
- DNS Servers: Servers used to resolve domain names.

11. Important concepts used in the outputs

Term	Meaning / Use
IP Address	Identifies a device on a network
Subnet Mask	Separates network and host portions
Default Gateway	Connects local network to other networks
DNS	Converts domain names into IP addresses
DHCP	Automatically assigns IP addresses
MAC Address	Hardware address of a network interface
TTL	Time To Live
Port Number	Identifies an application or service
Packet Loss	Data packets that fail to reach destination

12. Quick Revision Table

Command	Syntax	Main Use
IPConfig	ipconfig	IP configuration
Ping	ping google.com	Connectivity
Tracert	tracert google.com	Route tracking
Netstat	netstat	Active connections
Hostname	hostname	Computer name
Nslookup	nslookup google.com	DNS lookup
ARP	arp -a	IP to MAC mapping
Getmac	getmac	MAC address
Pathping	pathping google.com	Packet loss
IPConfig /all	ipconfig /all	Complete configuration

Important practical note: These are sample outputs. Your actual output may differ depending on your laptop, Wi-Fi connection and network configuration.

For your external lab: Learn the theory, syntax and output explanation for each command. If the examiner asks you to execute a command, open CMD, type the syntax and explain the displayed fields.

Computer Networks – CRC Python Programs

Simple lab-exam programs with sample outputs

How to use: Run each program separately. For the received data, first enter the transmitted data exactly to get “No error detected!”. To test error detection, change one bit in the received data.

Important: CRC-12, CRC-16 and CRC-CCITT use different generator polynomials. The CRC-16 example below uses the normal polynomial 0x8005; CRC-CCITT uses 0x1021.

Basic CRC

```
def divide(a, b):
    a = list(a)

    for i in range(len(a) - len(b) + 1):
        if a[i] == '1':
            for j in range(len(b)):
                a[i+j] = str(int(a[i+j]) ^ int(b[j]))

    return ''.join(a)

data = input('Enter data: ')
gen = input('Enter generator: ')

temp = data + '0' * (len(gen)-1)
result = divide(temp, gen)
crc = result[len(data):]

print('CRC:', crc)
transmitted = data + crc
print('Transmitted data:', transmitted)

received = input('Enter received data: ')
check = divide(received, gen)

if '1' in check[-(len(gen)-1):]:
    print('Error detected!')
else:
    print('No error detected!')
```

Sample output:

```
Enter data: 1101
Enter generator: 1011
CRC: 001
Transmitted data: 1101001
Enter received data: 1101001
No error detected!
```

CRC-12

```
def divide(a, b):
    a = list(a)

    for i in range(len(a) - len(b) + 1):
        if a[i] == '1':
            for j in range(len(b)):
                a[i+j] = str(int(a[i+j]) ^ int(b[j]))

    return ''.join(a)

data = input('Enter data: ')
gen = '1100000001111'

temp = data + '0' * (len(gen)-1)
result = divide(temp, gen)
crc = result[len(data):]

print('CRC:', crc)
transmitted = data + crc
print('Transmitted data:', transmitted)

received = input('Enter received data: ')
check = divide(received, gen)

if '1' in check[-(len(gen)-1):]:
    print('Error detected!')
```

```
else:
    print('No error detected!')
```

Sample output:

```
Enter data: 110101
CRC-12: 000111000010
Transmitted data: 110101000111000010
Enter received data: 110101000111000010
No error detected!
```

CRC-16 (CRC-16/IBM polynomial)

```
def divide(a, b):
    a = list(a)

    for i in range(len(a) - len(b) + 1):
        if a[i] == '1':
            for j in range(len(b)):
                a[i+j] = str(int(a[i+j]) ^ int(b[j]))

    return ''.join(a)

data = input('Enter data: ')
gen = '10000000000000101'

temp = data + '0' * (len(gen)-1)
result = divide(temp, gen)
crc = result[len(data):]

print('CRC:', crc)
transmitted = data + crc
print('Transmitted data:', transmitted)

received = input('Enter received data: ')
check = divide(received, gen)

if '1' in check[-(len(gen)-1):]:
    print('Error detected!')
else:
    print('No error detected!')
```

Sample output:

```
Enter data: 1011001
CRC-16: 0000000100111101
Transmitted data: 10110010000000100111101
Enter received data: 10110010000000100111101
No error detected!
```

CRC-CCITT (0x1021)

```
def divide(a, b):
    a = list(a)

    for i in range(len(a) - len(b) + 1):
        if a[i] == '1':
            for j in range(len(b)):
                a[i+j] = str(int(a[i+j]) ^ int(b[j]))

    return ''.join(a)

data = input('Enter data: ')
gen = '10001000000100001'

temp = data + '0' * (len(gen)-1)
result = divide(temp, gen)
crc = result[len(data):]

print('CRC:', crc)
transmitted = data + crc
print('Transmitted data:', transmitted)

received = input('Enter received data: ')
check = divide(received, gen)

if '1' in check[-(len(gen)-1):]:
    print('Error detected!')
else:
    print('No error detected!')
```

Sample output:

```
Enter data: 1011001
CRC-CCITT: 1100101111011100
Transmitted data: 10110011100101111011100
Enter received data: 10110011100101111011100
No error detected!
```

Quick revision

1. Add (generator length − 1) zeros to the data.
2. Perform modulo-2 division using XOR.
3. The remainder is the CRC.
4. Append CRC to the original data to form transmitted data.
5. Divide received data by the same generator. A zero remainder means no error is detected.

XOR rule: 0 XOR 0 = 0; 0 XOR 1 = 1; 1 XOR 0 = 1; 1 XOR 1 = 0.

Data Link Layer – Framing Methods

Three simple Python programs with sample inputs and outputs shown one line at a time.

1. Character Count

Enter frames separated by spaces. Each frame is prefixed with its length + 1.

```
frames = input("Enter frames: ").split()

stream = ""

for f in frames:
    stream += str(len(f) + 1) + f

print("Transmitted stream:", stream)

i = 0
while i < len(stream):
    n = int(stream[i])
    print("Frame:", stream[i+1:i+n])
    i += n
```

Sample input

```
ABC DE FG
```

Output (line by line)

```
Transmitted stream: 4ABC3DE3FG
Frame: ABC
Frame: DE
Frame: FG
```

2. Character Stuffing (Byte Stuffing)

ESC is added before FLAG or ESC when either appears inside the data. FLAG marks the frame boundaries.

```
data = input("Enter data: ").split()

stuffed = ["FLAG"]

for ch in data:
    if ch == "FLAG" or ch == "ESC":
        stuffed.append("ESC")
    stuffed.append(ch)

stuffed.append("FLAG")

print("Stuffed frame:", " ".join(stuffed))

result = []
i = 1

while i < len(stuffed) - 1:
    if stuffed[i] == "ESC":
        i += 1

    result.append(stuffed[i])
    i += 1

print("Destuffed data:", " ".join(result))
```

Sample input

```
A B FLAG C ESC D
```

Output (line by line)

```
Stuffed frame: FLAG A B ESC FLAG C ESC ESC D FLAG
Destuffed data: A B FLAG C ESC D
```

3. Bit Stuffing

A 0 is inserted after every five consecutive 1s. The frame delimiter is 01111110.

```
bits = input("Enter bits: ")

stuffed = ""
count = 0

for b in bits:
    stuffed += b

    if b == "1":
        count += 1
        if count == 5:
            stuffed += "0"
            count = 0
    else:
        count = 0

frame = "01111110" + stuffed + "01111110"
print("Stuffed frame:", frame)

data = frame[8:-8]
result = ""
count = 0
i = 0

while i < len(data):
    result += data[i]

    if data[i] == "1":
        count += 1
        if count == 5:
            i += 1
            count = 0
    else:
        count = 0

    i += 1

print("Destuffed bits:", result)
```

Sample input

```
11111011111
```

Output (line by line)

```
Stuffed frame: 01111110111110011111001111110
Destuffed bits: 11111011111
```

Quick revision

```
Character Count     : Length + 1
Character Stuffing  : Add ESC before FLAG / ESC
Bit Stuffing         : Add 0 after five consecutive 1s
```